Question (advanced): "How would you design a functional data processing pipeline that also interacts with class-based modules?"

Analysis: This question evaluates a candidate's ability to architect hybrid systems that combine functional programming and object-oriented programming paradigms effectively.

Answer: Designing a functional data processing pipeline that integrates with class-based modules involves these steps:

1. *Functional core*—Build the core pipeline using pure, composable functions for all data transformations. Each function should:
    - Take an input and return an output without side effects.
    - Avoid mutating external states or relying on variables outside its scope.
      This approach ensures predictability, testability, and referential transparency.
2. *Object-oriented shell*—Encapsulate all stateful operations and interactions with external systems in class-based modules, such as:
    - Fetching raw data (API calls, database queries).
    - Saving processed results (file systems, external APIs).
      These classes act as adapters, isolating impure operations from the functional core.
3. *Pipeline composition*—Use a utility like `pipe()` or `compose()` to declaratively chain functions together. This creates a clear, maintainable data flow, where each step is modular and easy to test in isolation.

Listing 3.8 demonstrates the example code in which the functional pipeline is composed by chaining pure functions. The pipeline receives raw data from a class (such as `DataSource`), processes it through a sequence of functional transformations, and then passes the final result to another class (such as `DataSink`) for storage or output. This clear separation of concerns enhances modularity and makes the system easier to maintain and extend.

Listing 3.8 A functional data processing pipeline interacting with class-based modules

```
const apiClient = { #A
  async get(endpoint) {
```